

为什么要使用 GitLab?

GitLab 是集中服务器上管理 git 存储库的一个好方法。GitLab 让您完全可以控制您的存储库或项目，并允许您自己决定是公共还是私有。

特征

- GitLab 免费托管您的(私人)软件项目。
- GitLab 是管理 Git 存储库的平台。
- GitLab 提供免费的公共和私人存储库，问题跟踪和维基。
- GitLab 是 Git 之上的一个用户友好的 Web 界面层，它提高了使用 Git 的速度。
- GitLab 提供了自己的持续集成(CI)系统来管理项目，并提供用户界面以及 GitLab 的其他功能。

优点

- GitLab 提供了 GitLab Community Edition 版本，供用户在他们的代码所在的服务器上定位。
- GitLab 免费提供无限数量的私人 and 公共存储库。
- 代码片段可以共享项目中的少量代码，而不是共享整个项目。

缺点

- 在推拉(push/pull)仓库时，它并不像 GitHub 那么快。
- 从一个页面切换到另一个页面时，GitLab 界面需要时间。

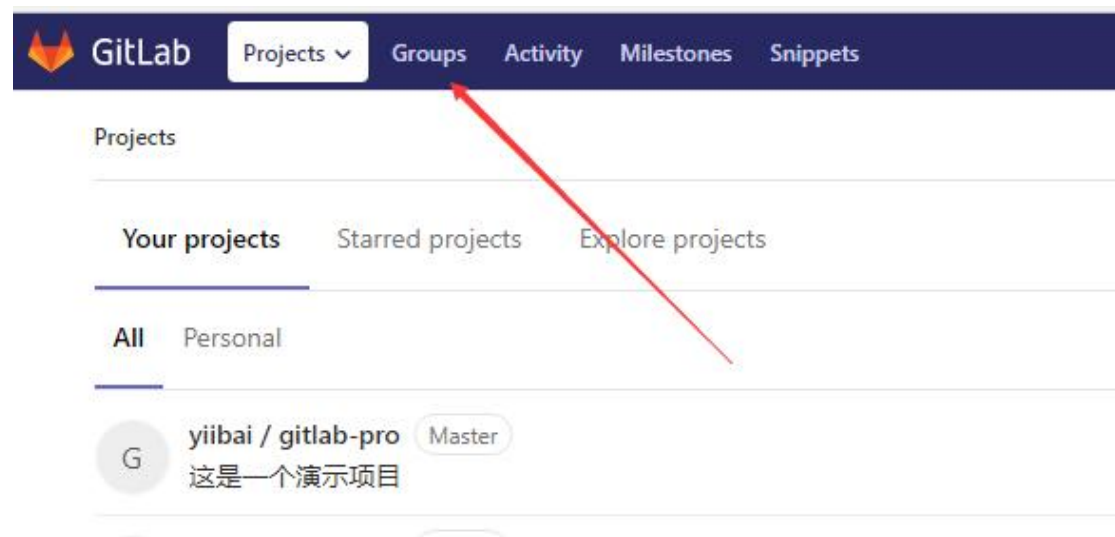
GitLab 用户和组

GitLab 创建组

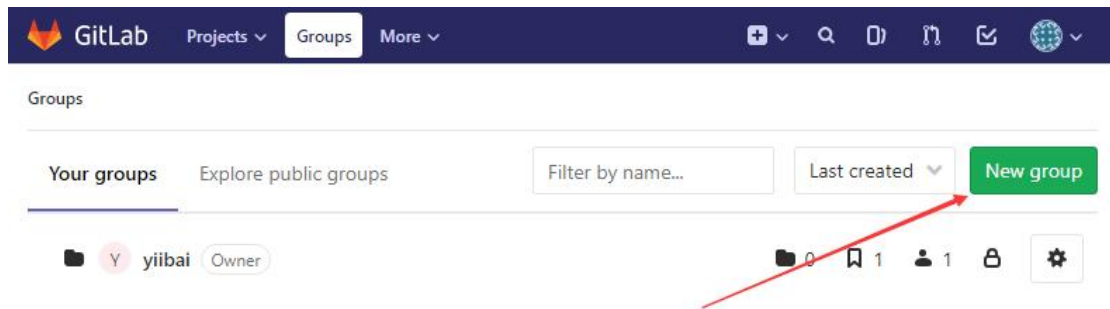
创建组有助于连接多个存储库，并允许成员通过授予组级别的权限来访问项目。

创建组的步骤

步骤(1): 登录到您的 GitLab 帐户，然后单击 Group 菜单:



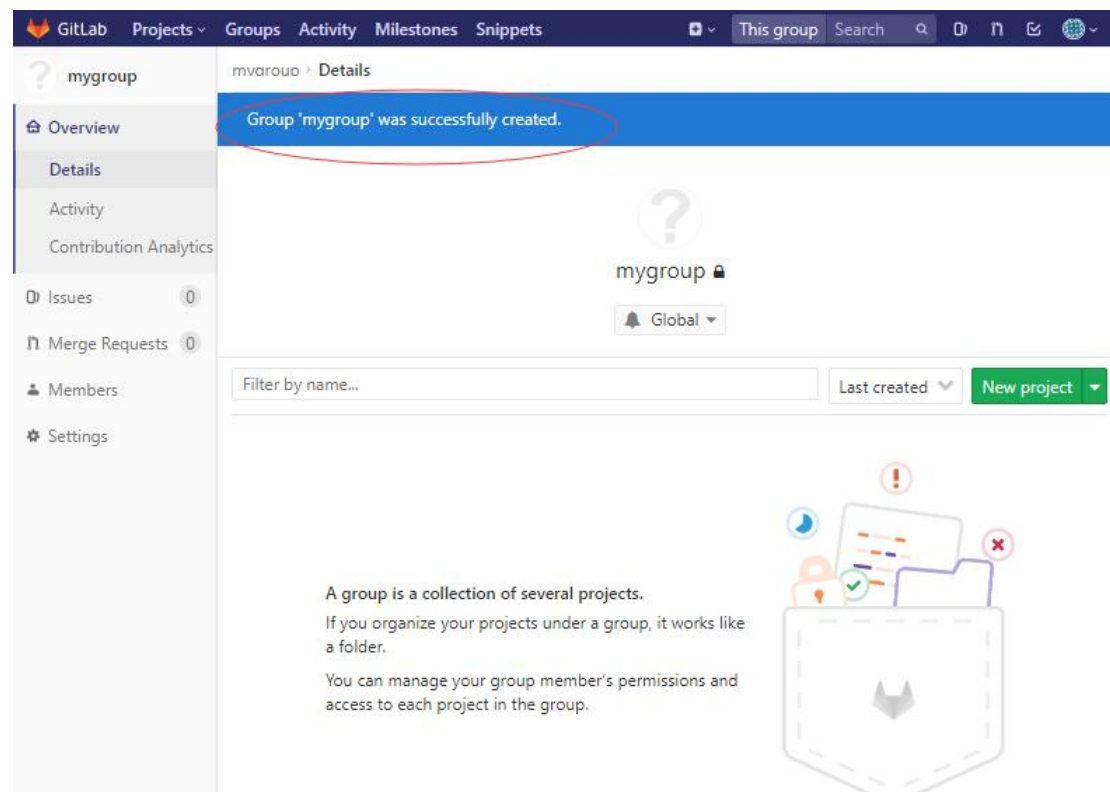
步骤(2): 接下来, 您将看到下面的屏幕并单击 New group 按钮来创建组:



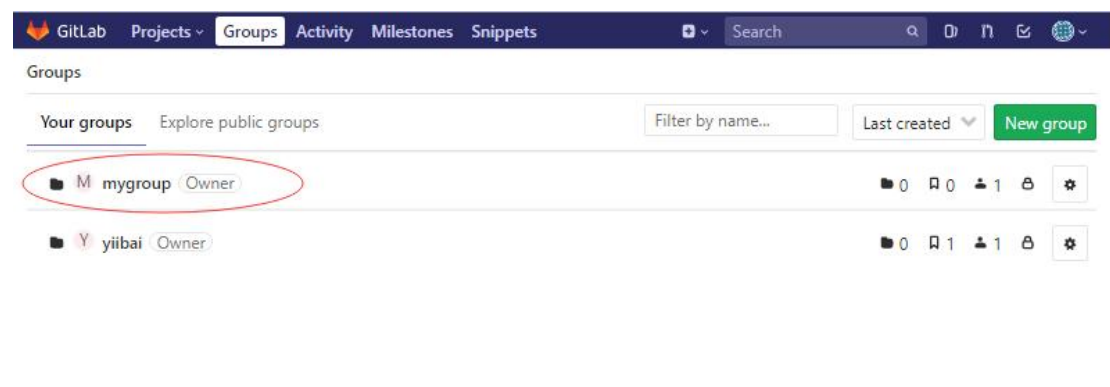
步骤(3): 输入组名称, 描述, 可见性级别(私有/公共/内部), 并且您可以设置组的图像, 大小应该在 200kb 以内。 现在点击 Create group 按钮。

A screenshot of the 'New Group' form in GitLab. The form is titled 'New Group' and contains several input fields and options. The 'Group path' field is pre-filled with 'https://gitlab.com/' and 'mygroup'. The 'Group name' field is also 'mygroup'. The 'Description' field is empty. The 'Group avatar' section has a 'Choose File ...' button and a note 'No file chosen' with a sub-note 'The maximum file size allowed is 200KB.'. The 'Visibility Level' section has three radio buttons: 'Private' (selected), 'Internal', and 'Public'. Below these are explanatory text blocks for each level. At the bottom, there are two buttons: 'Create group' (green) and 'Cancel' (grey).

步骤(4): 接下来, 它将在创建组后显示成功消息, 如下所示:



步骤(5): 现在, 返回到您的 Groups 部分, 将在列表中看到创建的组:

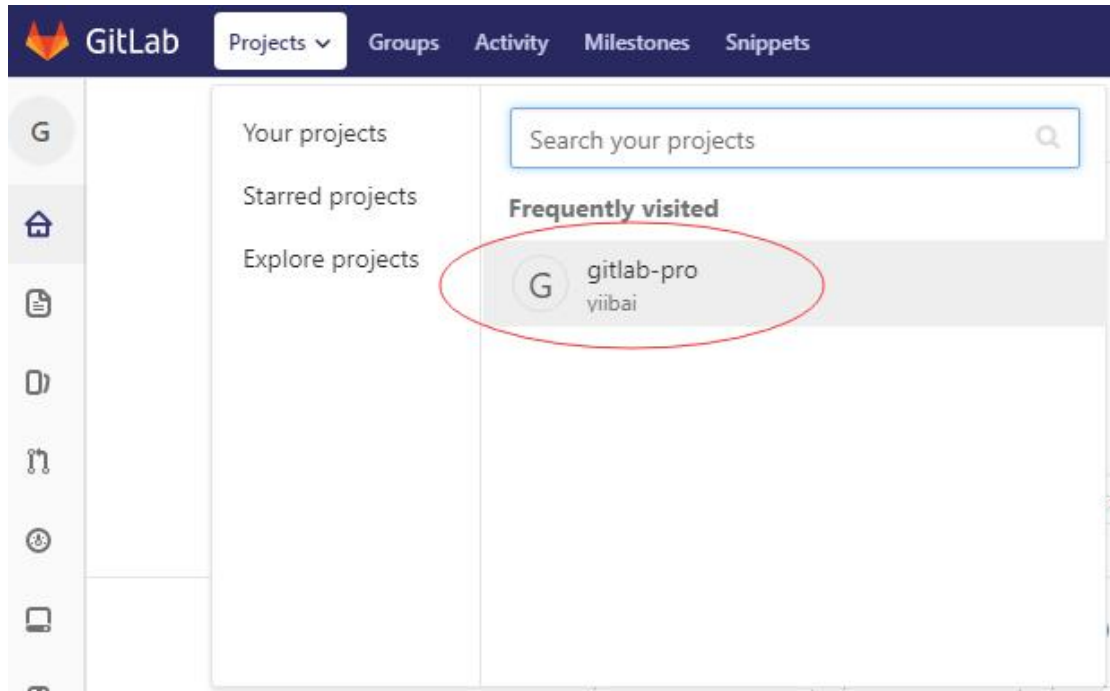


GitLab 创建用户

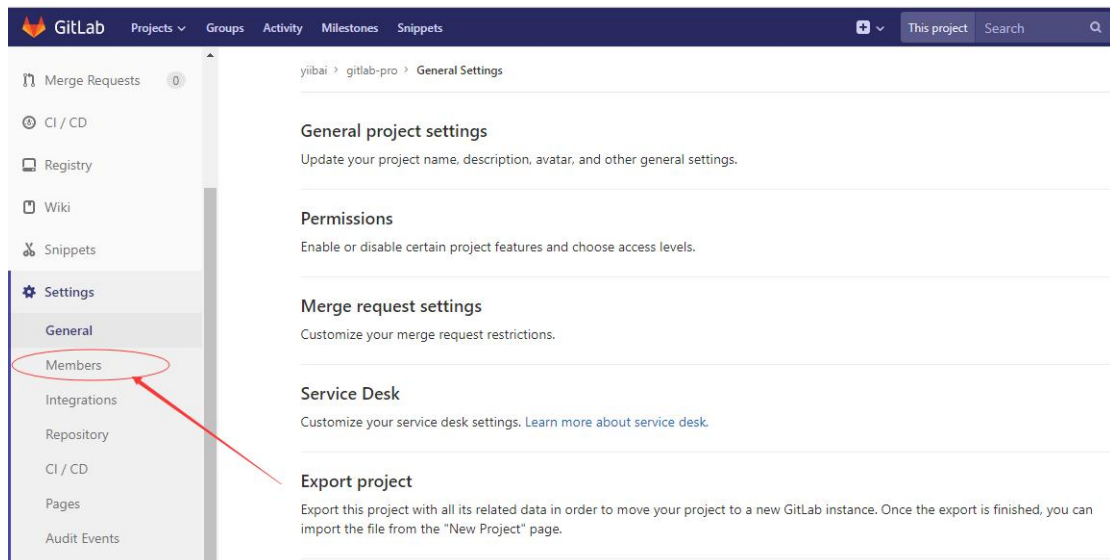
在本章中, 我们将讨论如何将用户添加到 GitLab 中的项目中。

添加用户的步骤

步骤(1):登录到您的 GitLab 帐户并转到 Projects 部分下。如下图所示 -



步骤(2): 接下来，单击设置选项卡下的 Members 选项:



步骤(3): 它会打开下面的界面，用于将成员添加到您的项目中:

The screenshot shows the GitLab web interface for adding a new member to a project. The left sidebar contains navigation links: Overview, Repository, Issues (0), Merge Requests (0), CI / CD, Registry, Wiki, Snippets, Settings (General, Members, Integrations), and a Collapse sidebar button. The main content area is titled 'Project members' and includes a sub-header 'You can add a new member to **gitlab-pro** or share it with another group.' Below this is a form with the following sections: 'Add member' (a button), 'Select members to invite' (a search input field), 'Choose a role permission' (a dropdown menu set to 'Guest' with a link to 'Read more about role permissions'), and 'Access expiration date' (an input field for 'Expiration date'). At the bottom of the form are two buttons: 'Add to project' and 'Import'. Below the form, there is a section titled 'Existing members and groups' showing 'Members of **gitlab-pro** 1'.

步骤(4): 现在输入用户名称，角色权限，到期日期(可选)，然后点击 Add to project 按钮将用户添加到项目中:

This screenshot is an annotated version of the 'Add member' form. A red circle highlights the 'test x' text entered in the 'Select members to invite' search field. A red arrow points from the 'Add to project' button to the 'Expiration date' input field, indicating that the expiration date is optional. The rest of the form, including the 'Guest' role selection and the 'Import' button, is visible but not annotated.

步骤(5): 接下来，在将用户添加到项目后，您将收到一条成功的消息。

yiibai > gitlab-pro > Members

Users were successfully added.

Project members
You can add a new member to **gitlab-pro** or share it with another group.

Add member	Share with group
Select members to invite Search for members to update or invite Choose a role permission Guest Read more about role permissions Access expiration date Expiration date Add to project Import	

上图中突出显示的框表示，新项目已添加到项目中：

Existing members and groups

Members of **gitlab-pro** 2

Find existing members by name

Name, ascending

 test @testchallenge
Joined 3 minutes ago

Guest

Expiration date

 yiibai @yiibai
Joined 3 days ago

It's you

Master

步骤(6):您也可以通过单击 “import” 按钮将用户添加到项目中：

Project members
You can add a new member to **gitlab-pro** or share it with another group.

Add member	Share with group
Select members to invite Search for members to update or invite Choose a role permission Guest Read more about role permissions Access expiration date Expiration date Add to project Import	

步骤(7): 现在选择您要将用户添加到项目中的项目, 然后单击 Import project members 按钮:

yiibai > gitlab-pro > Import members

Import members from another project
Only project members will be imported. Group members will be skipped.

Project

步骤(8): 将用户导入到项目后, 您将收到成功消息:

yiibai > gitlab-pro > Members

Successfully imported

Project members
You can add a new member to **gitlab-pro** or share it with another group.

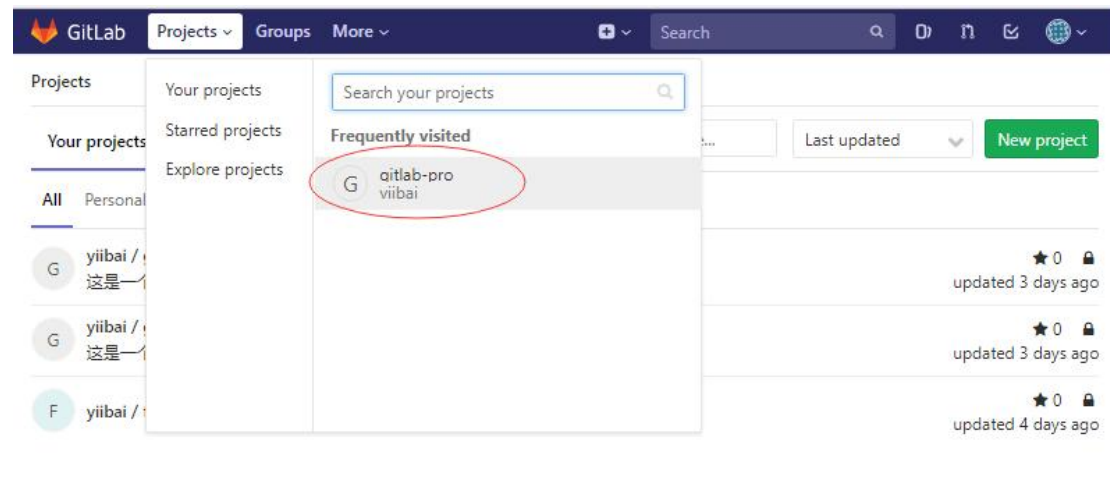
Add member	Share with group
Select members to invite Search for members to update or invite	
Choose a role permission Guest Read more about role permissions	
Access expiration date Expiration date	
Add to project	Import

GitLab 删除用户

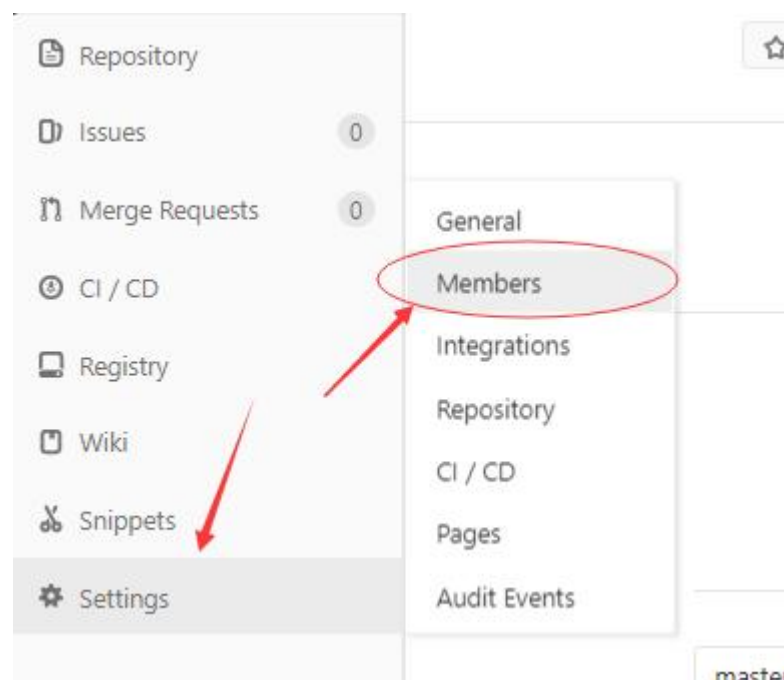
在本章中, 我们将讨论如何从 GitLab 中的项目中删除用户。

删除用户的步骤

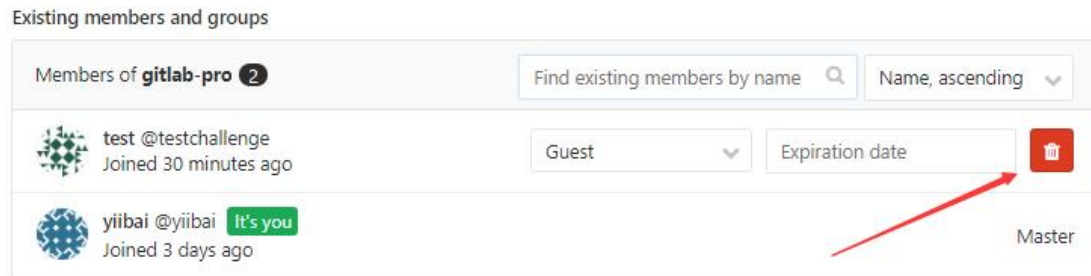
步骤(1): 登录到您的 GitLab 帐户并转到 Projects 部分下:



步骤(2): 现在, 单击 Settings 选项卡下的 Members 选项:



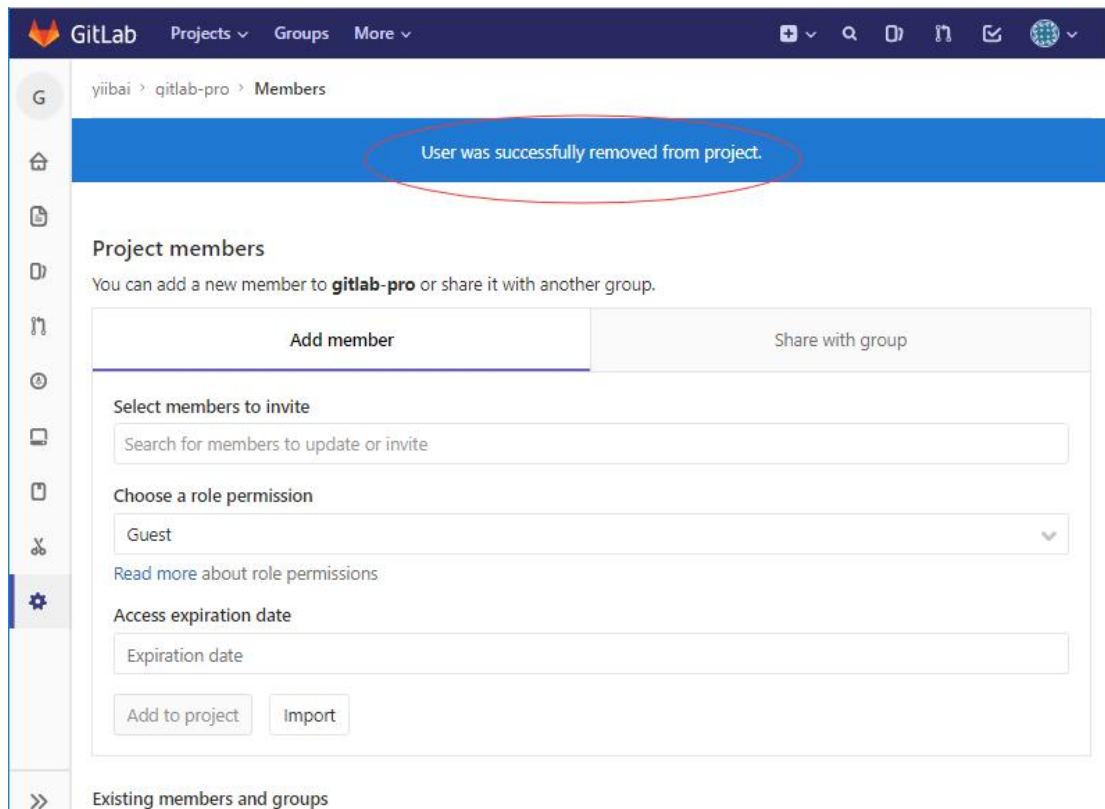
步骤(3): 您将看到 “Existing members and groups” 部分下的用户列表，并单击右侧的删除选项以从项目中删除用户：



步骤(4): 点击删除按钮后，会弹出一个窗口，提示是否从项目中删除选中的用户。 点击确定按钮删除用户。



步骤(5): 现在，从项目中删除用户后，它将显示成功消息，如下图所示：



GitLab Git 命令使用

Git 命令用于与其他开发人员轻松共享和组合代码。

Git 命令

以下是一些基本 Git 命令：

可以使用以下命令来检查 Git 的版本：

```
$ git --version
```

在提交信息时添加 Git 用户名和电子邮件地址以标识作者。 使用以下

命令设置用户名：

```
$ git config --global user.name "Yiibai"
```

输入用户名后，使用以下命令验证输入的用户名：

```
$ git config --global user.name
```

接下来，使用以下命令设置电子邮件地址：

```
$ git config --global user.email "email_address@yiibai.com"
```

可以验证输入的电子邮件地址：

```
$ git config --global user.email
```

使用以下命令检查输入的信息：

```
$ git config --global --list
```

可以使用以下命令来获取对主分支所做的最新更改：

```
$ git checkout master
```

可以使用以下命令获取工作目录的最新更改：

```
$ git pull origin NAME-OF-BRANCH -u
```

在这里，NAME-OF-BRANCH 可以是 'master' 或任何其他现有的分支。

用下面的命令创建一个新的分支:

```
$ git checkout -b branch-name
```

可以使用以下命令从一个分支切换到其他分支:

```
$ git checkout branch-name
```

使用以下命令检查对文件所做的更改:

```
$ git status
```

将看到红色的变化并将文件添加到 staging 中, 如下所示:

```
$ git add file-name
```

或者可以将所有文件添加到 staging 中, 如下所示:

```
$ git add *
```

现在使用以下命令将您的更改发送到 master 分支:

```
$ git push origin branch-name
```

通过使用以下命令删除所有更改, 但未暂挂的东西除外:

```
$ git checkout .
```

可以使用以下命令删除未跟踪文件的所有更改:

```
$ git clean -f
```

要将不同分支与主分支合并，请使用以下命令：

```
$ git checkout branch-name
```

```
$ git merge master
```

也可以使用以下命令将主分支与创建的分支合并：

```
$ git checkout master
```

```
$ git merge branch-name
```

GitLab SSH 密钥设置

SSH 代表用于管理网络，操作系统和配置的 Secure Shell 或 Secure Socket Shell，并且每次都不需要使用用户名和密码即可验证 GitLab 服务器。您可以设置 SSH 密钥以提供计算机与 GitLab 之间的可靠连接。在生成 ssh keygen 之前，您需要在系统中安装 Git。

创建 SSH 密钥

步骤(1)：要创建 SSH 密钥，请打开 Git Bash 命令提示符并输入命令，如下所示：

```
ssh-keygen
```

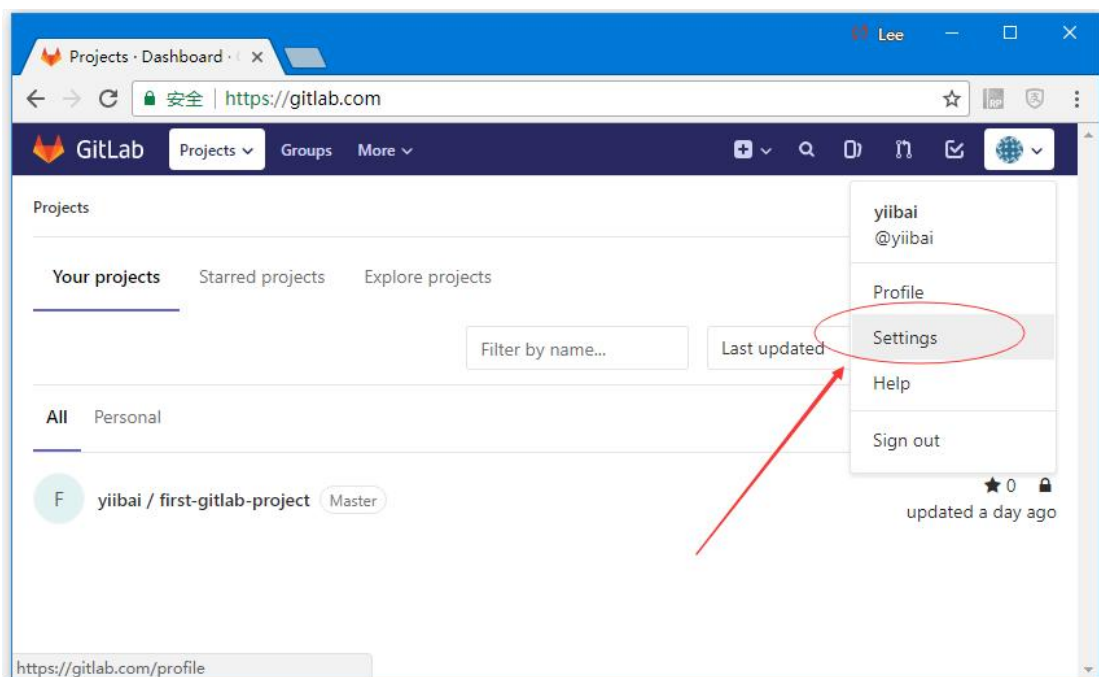
它会提示“输入保存密钥的文件(//.ssh/id_rsa):”，只需键入文件名并按回车。接下来提示输入密码显示“输入密码(空密码)”。输入一些密码并按回车。您将看到生成的 SSH 密钥，如下图所示：

```

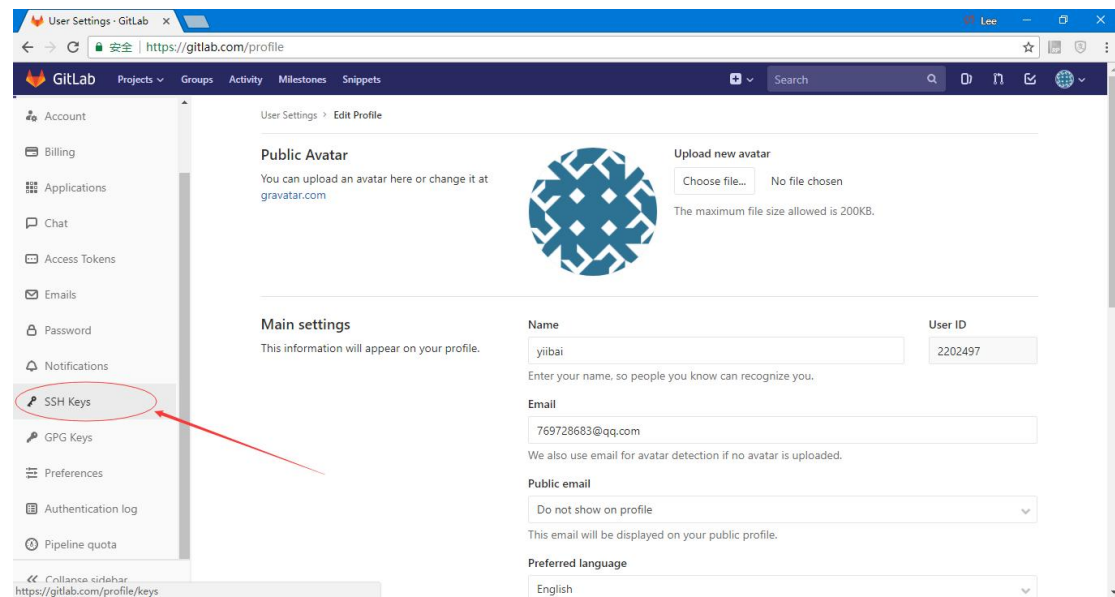
MINGW32/C:/Users/Administrator/Desktop
Administrator@MY-PC /C:/Users/Administrator/Desktop (master)
$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/c:/Users/Administrator/.ssh/id_rsa):
/c:/Users/Administrator/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /c:/Users/Administrator/.ssh/id_rsa.
Your public key has been saved in /c:/Users/Administrator/.ssh/id_rsa.pub.
The key fingerprint is:
3c:fc:34:94:68:55:bd:65:68:3c:36:8e:fa:a4:a0:a2 Administrator@MY-PC
The key's randomart image is:
+--[ RSA 2048 ]-----+
|      . . . . .      |
|      o .   O o      |
|     o o   = *       |
|    + . .   o        |
|    S o.             |
|    .+...           |
|    . . .+          |
|    . . . .         |
|    E. .             |
+-----+
$
Administrator@MY-PC /C:/Users/Administrator/Desktop (master)
$

```

步骤(2): 现在登录到您的 GitLab 帐户, 然后单击[Settings] 选项。



步骤(3): 要创建 SSH 密钥, 请单击菜单左侧的 SSH Key 选项卡。



步骤(4): 现在转到您的电脑 C 盘驱动器, 您将看到第一步中生成的.pub 扩展名的文件(在目录:C:\Users\Administrator\.ssh)。

id_rsa	2018/4/12 9:08	文件	2 KB
id_rsa.pub	2018/4/12 9:08	PUB 文件	1 KB
known_hosts	2017/4/30 20:37	文件	3 KB

步骤(5): 接下来打开 `id_rsa.pub` 文件, 复制 SSH 密钥并将其粘贴到高亮显示的密钥框中, 如下图所示:

User Settings > SSH Keys

SSH Keys

SSH keys allow you to establish a secure connection between your computer and GitLab.

Add an SSH key

Before you can add an SSH key you need to [generate one](#) or use an [existing key](#).

Key

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCAUQ9r17PwJS/rUGhc1...b3QdF5b3AyAiMvw7+va
yC4hn2N/42rSe35v3MazpgHR0CoplB50VJwGT3zxFU0sPZUdn0+bFwbTrLdEKtd+GiiO0UkGPfwg8
r0jTk+Db+fsG+zA7jr17HS2s3i1TJJ3rbCSwjG...L5GQj32ni+JK6x0Uk...f6n0rjt
nyY1a5bVzEIsqtaV9WudbJr9AOG6wzVqbcOX+f6ilSqb087G2uj73
```

Title

Administrator@MY-PC

Add key

Your SSH keys (0)

There are no SSH keys with access to your account.

步骤(6): 单击添加密钥(Add key)按钮, 将 SSH 密钥添加到您的 GitLab。

您将看到 SSH 密钥的简短版本, 标题和创建日期, 如下图所示:

User Settings > SSH Keys > Administrator@MY-PC

SSH Key

Title: **Administrator@MY-PC**

Created on: **Apr 12, 2018 9:23am**

Last used on: **N/A**

Fingerprint: 3c:fc:34:94:68:55:bd:65:68:3c:36:8e:fa:a4:a0:a2

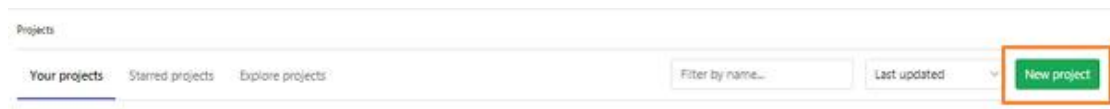
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQAC9r/hPwIS/cuGhsWUOHXkntnpbRGdF5b3AyA1Mvw7+vayC4hl

Remove

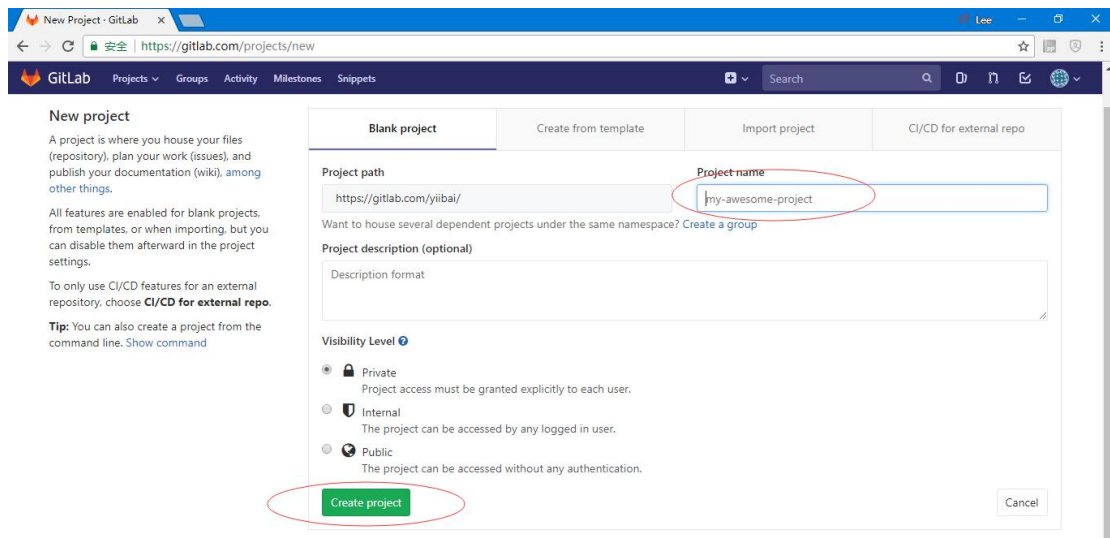
GitLab 创建项目

在本章中，我们将学习如何在 GitLab 中创建一个新项目。

步骤(1): 要创建新项目, 请登录到您的 GitLab 帐户, 然后单击仪表板中的新建项目(New project)按钮:

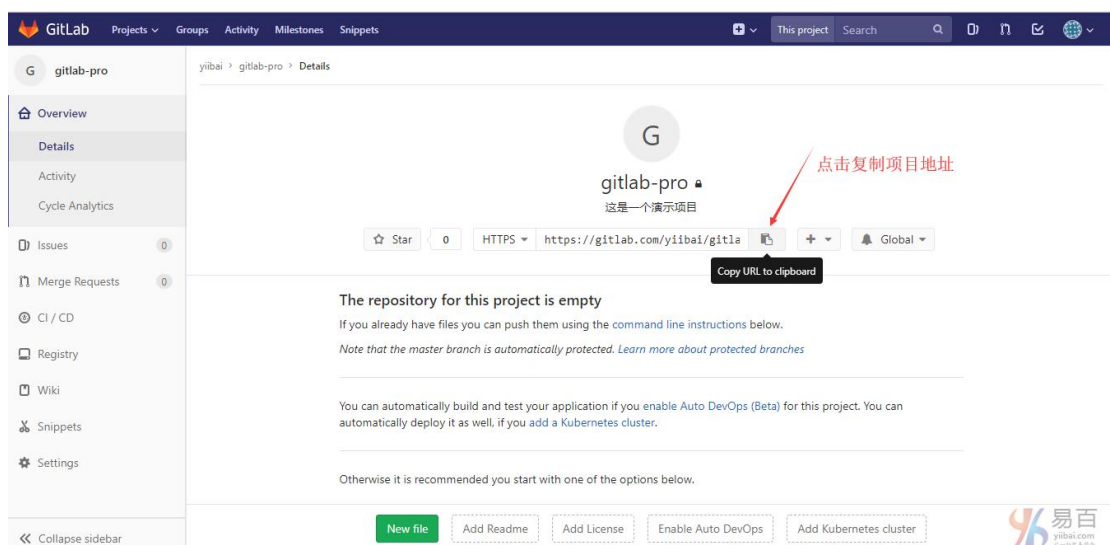


步骤(2): 它将打开新建项目界面, 如下图所示:



输入项目名称(假设项目名称为 gitlab-pro), 项目描述, 可见性级别(公开或内部访问项目的可见性), 然后单击创建项目(Create project)按钮。

步骤(3): 接下来, 它将创建一个新项目, 其成功消息如下所示:

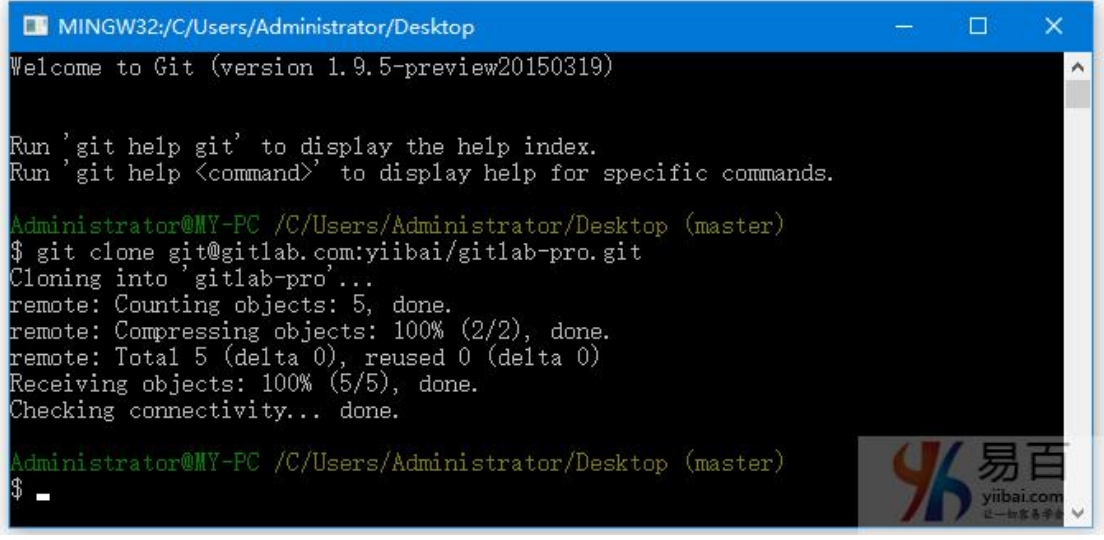


将存储库推送到项目

步骤(4): 您可以使用 `git-clone` 命令将存储库克隆到本地系统。在一个目录下, 打开 Git Bash, 执行以下命令:

```
$ git clone git@gitlab.com:yiibai/gitlab-pro.git
```

演示如下图所示 -

A screenshot of a Windows command prompt window titled "MINGW32/C:/Users/Administrator/Desktop". The window shows the output of the 'git clone' command. The text in the terminal is as follows:

```
Welcome to Git (version 1.9.5-preview20150319)

Run 'git help git' to display the help index.
Run 'git help <command>' to display help for specific commands.

Administrator@MY-PC /C:/Users/Administrator/Desktop (master)
$ git clone git@gitlab.com:yiibai/gitlab-pro.git
Cloning into 'gitlab-pro'...
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (5/5), done.
Checking connectivity... done.

Administrator@MY-PC /C:/Users/Administrator/Desktop (master)
$
```

A watermark for "易百" (yiibai.com) is visible in the bottom right corner of the terminal window.

`clone` 命令将库的副本放入一个名称为 `gitlab-pro` 的新目录中。

步骤(5): 现在转到您新创建的目录并输入以下命令:

```
$ cd gitlab-pro/
```

```
Administrator@MY-PC
```

```
/C:/Users/Administrator/Desktop/gitlab-pro (master)
```

```
$ touch README.md
```

上述命令将创建一个 README.md 文件，可以在其中放置有关文件夹的信息。

步骤(6): 使用以下命令将 README.md 文件添加到创建的目录中:

```
$ git add README.md
```

步骤(7): 现在将更改与日志消息一起存储，如下所示:

```
$ git commit -m "add README"
```

标志 -m 用于在提交中添加消息。

步骤(8): 将提交到本地分支上的远程仓库:

```
$ git push -u origin master
```

下图描述了在将提交提交到远程存储库时使用上述命令的情况:

```
MINGW32:/C:/Users/Administrator/Desktop/gitlab-pro
remote: Counting objects: 5, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (5/5), done.
Checking connectivity... done.

Administrator@MY-PC /C:/Users/Administrator/Desktop (master)
$ cd gitlab-pro/

Administrator@MY-PC /C:/Users/Administrator/Desktop/gitlab-pro (master)
$ touch README.md

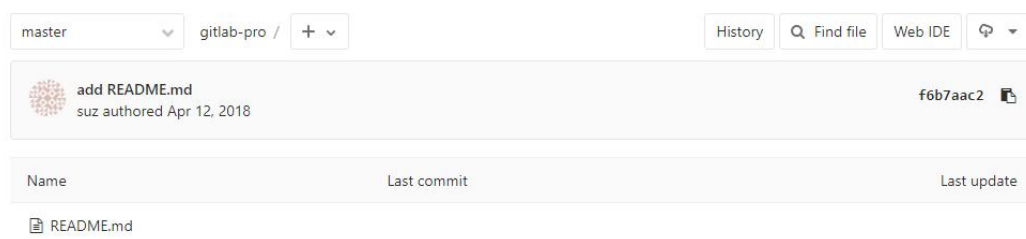
Administrator@MY-PC /C:/Users/Administrator/Desktop/gitlab-pro (master)
$ git add README.md

Administrator@MY-PC /C:/Users/Administrator/Desktop/gitlab-pro (master)
$ git commit -m "add README.md"
[master f6b7aac] add README.md
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md

Administrator@MY-PC /C:/Users/Administrator/Desktop/gitlab-pro (master)
$ git push -u origin master
Counting objects: 4, done.
Writing objects: 100% (3/3), 246 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To git@gitlab.com:yiibai/gitlab-pro.git
 a44a8c3..f6b7aac master -> master
Branch master set up to track remote branch master from origin.

Administrator@MY-PC /C:/Users/Administrator/Desktop/gitlab-pro (master)
$
```

浏览项目：<https://gitlab.com/yiibai/gitlab-pro>，可以看到有以下添加的文件：



GitLab 分叉项目

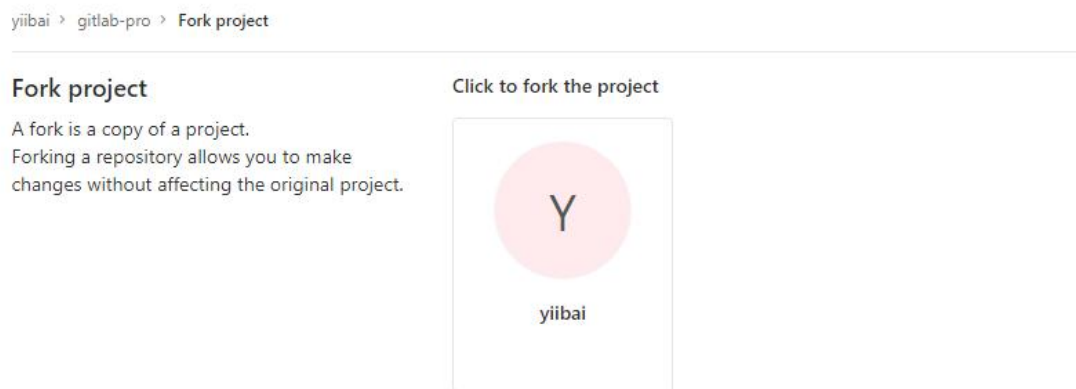
分叉(Fork)是原始存储库的一个副本，您可以在不影响原始项目的情况下进行更改。

分叉项目

步骤(1): 要分叉一个项目, 请项目详细下面单击 Fork 按钮, 以上节中创建的 gitlab-pro 项目为例, 如下所示:



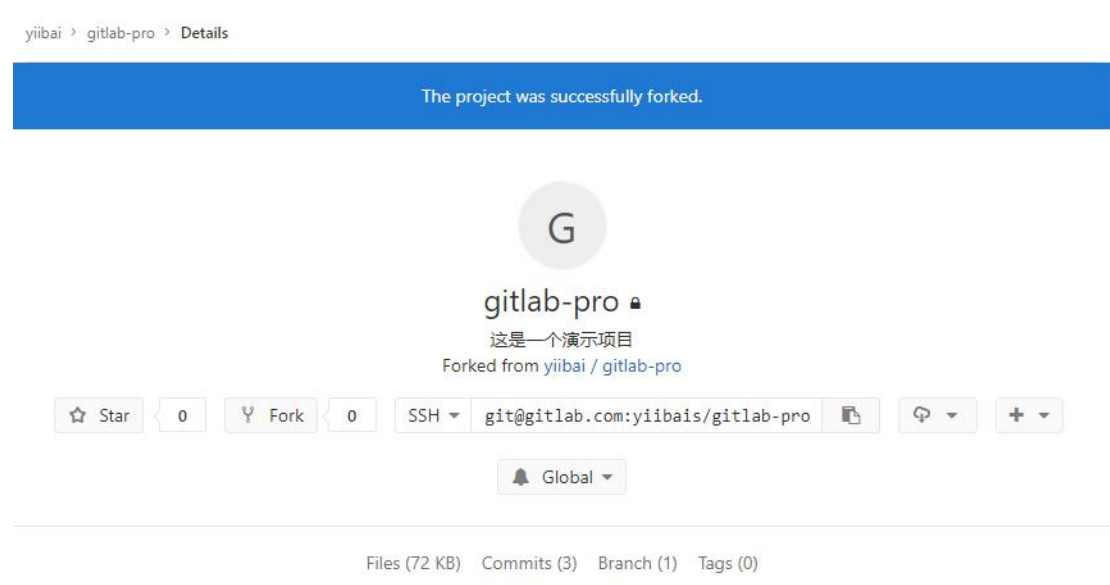
步骤(2): 在分叉项目之后, 需要通过单击将分叉的项目添加到分支组:



注意:如果提示没有命名空间权限(namespace), 可以先创建一个分组后, 再创建分叉。

步骤(3): 接下来, 它将开始处理项目一段时间。

步骤(4): 分叉项目过程完成后显示成功消息:

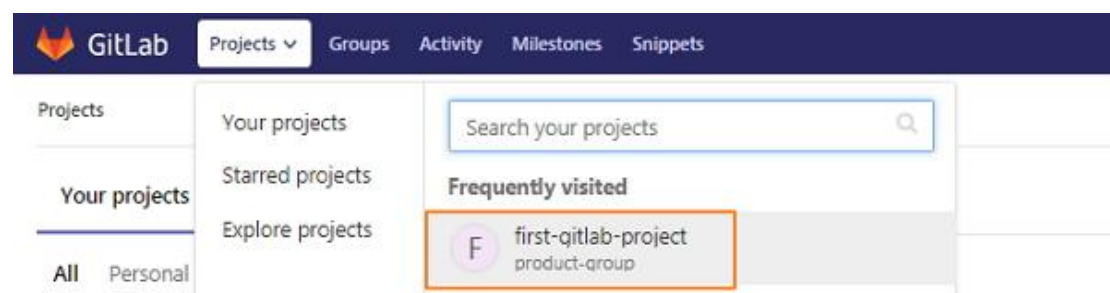


GitLab 分叉项目

分支是独立的线路和开发过程的一部分。 分支的创建包括以下步骤。

创建一个分支

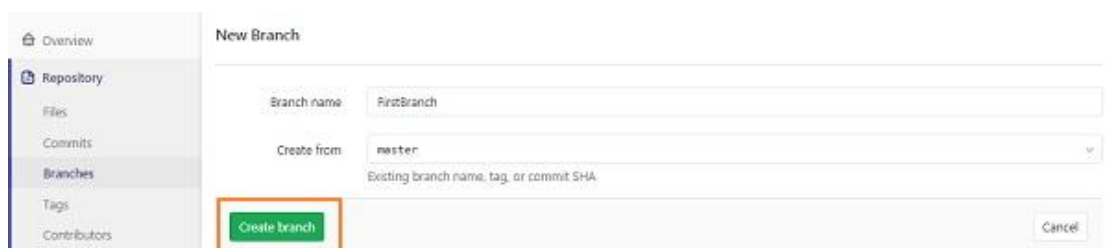
步骤(1): 登录到您的 GitLab 帐户并转到项目部分下的项目。



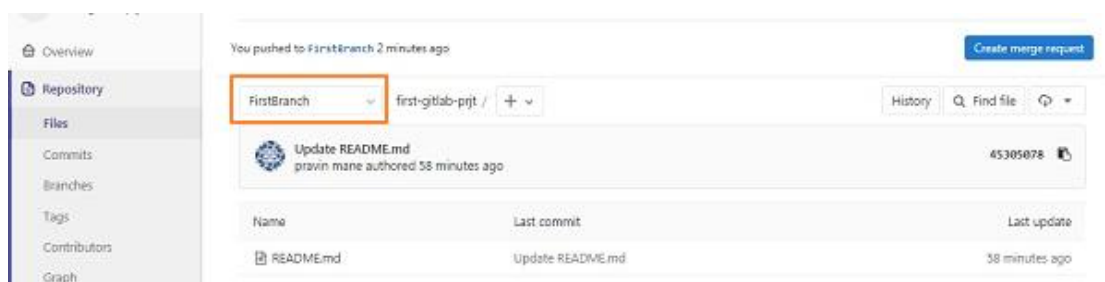
步骤(2): 要创建分支, 请单击 “Repository” 部分下的 Branches 选项, 然后单击 “New branch” 按钮。



步骤(3): 在 New branch 界面中, 输入分支的名称, 然后单击 Create branch 按钮。



步骤(4): 创建分支后, 您将会看到下面创建的分支。



GitLab Rebase 操作

当您使用长时间运行的分支时, Rebase 是一种将主合并到分支的方式。

Rebase 操作的步骤

步骤(1): 进入您的项目目录并使用 `git checkout` 命令创建一个名称为 `rebase-example` 的新分支:


```
$ git checkout -b rebase-example
```

标志-b 表示新的分支名称。

步骤(2): 现在, 创建一个新文件并向该文件添加一些内容, 如下所示:

```
$ echo "welcome to yiibai.com " >> rebase-file.md
```

"welcome to yiibai.com " 内容将被添加到 rebase-file.md 文件中。

步骤(3): 将新文件添加到工作目录, 并将更改与消息(通过使用 git commit 命令)一起存储到存储库中, 如下所示:

```
git commit . -m "Rebase file added"
```

标志-m 用于在提交中添加消息。

步骤(4): 现在, 切换到'master'分支。可以使用 git checkout 命令获取远程分支(master 是分支名称):

```
$ git checkout master
```

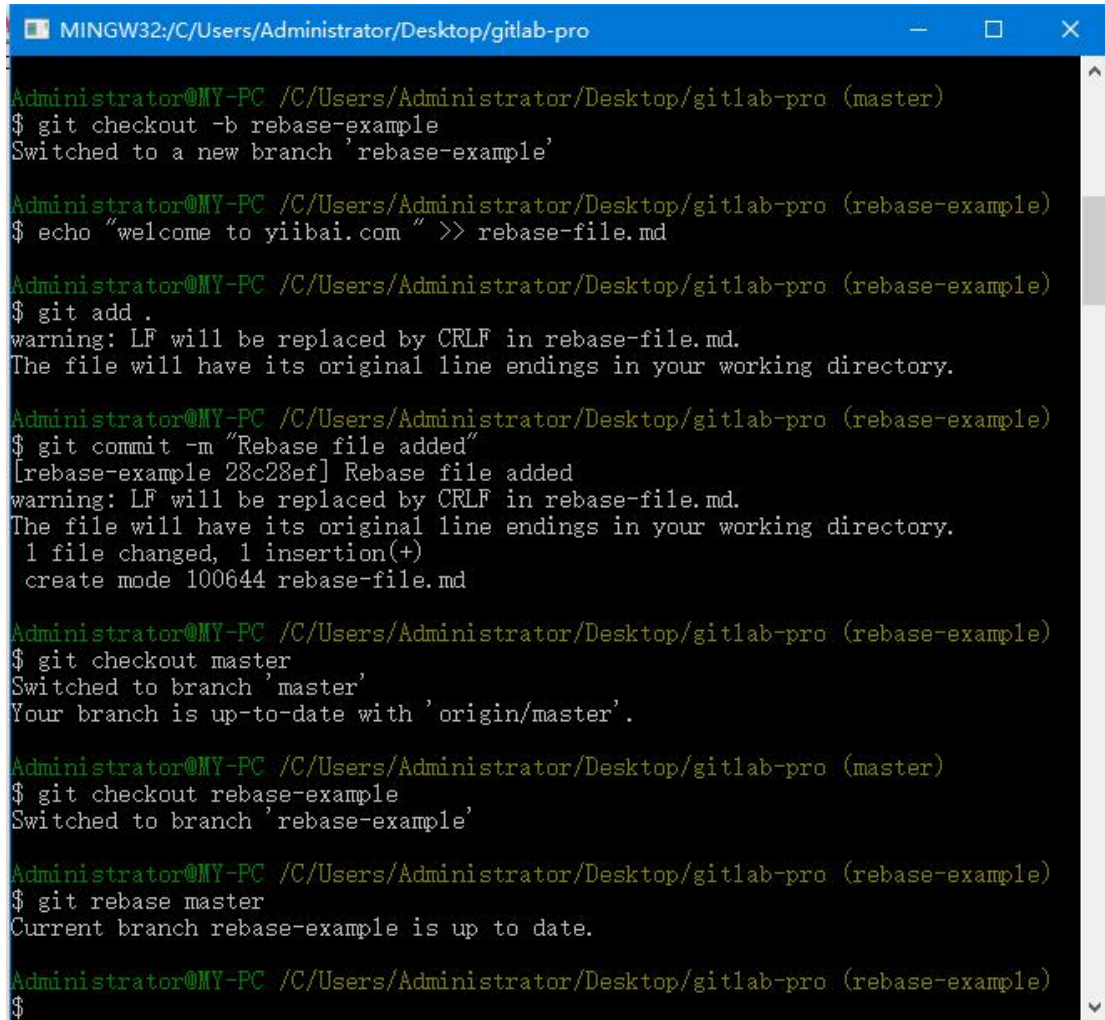
步骤(6): 切换到 rebase-example 分支以提交 master 分支。

```
$ git checkout rebase-example
```

步骤(7): 现在, 可以使用 git rebase 命令将主分支提交到 rebase-example 分支:

```
$ git rebase master
```

完整的操作过程如下图所示 -



```
Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (master)
$ git checkout -b rebase-example
Switched to a new branch 'rebase-example'

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$ echo "welcome to yiibai.com" >> rebase-file.md

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$ git add .
warning: LF will be replaced by CRLF in rebase-file.md.
The file will have its original line endings in your working directory.

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$ git commit -m "Rebase file added"
[rebase-example 28c28ef] Rebase file added
warning: LF will be replaced by CRLF in rebase-file.md.
The file will have its original line endings in your working directory.
1 file changed, 1 insertion(+)
create mode 100644 rebase-file.md

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$ git checkout master
Switched to branch 'master'
Your branch is up-to-date with 'origin/master'.

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (master)
$ git checkout rebase-example
Switched to branch 'rebase-example'

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$ git rebase master
Current branch rebase-example is up to date.

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (rebase-example)
$
```

GitLab 压缩提交

当您获取合并请求时，压缩是将所有提交合并为一个的一种方式。

压缩提交的步骤

步骤(1): 转到您的项目目录, 使用 `git checkout` 命令检出名为 `squash-chapter` 的新分支:

```
$ git checkout -b squash-chapter
```

标志 `-b` 表示新的分支名称。

步骤(2): 现在, 创建一个包含两个提交的新文件, 将该文件添加到工作目录, 并将更改与提交消息一起存储到存储库中, 如下所示:

```
$ echo "Message 1" >> README.md
```

```
$ git add .
```

```
$ git commit . -m "committed message1"
```

然后, 再次操作 -

```
$ echo "Message 2" >> README.md
```

```
$ git add .
```

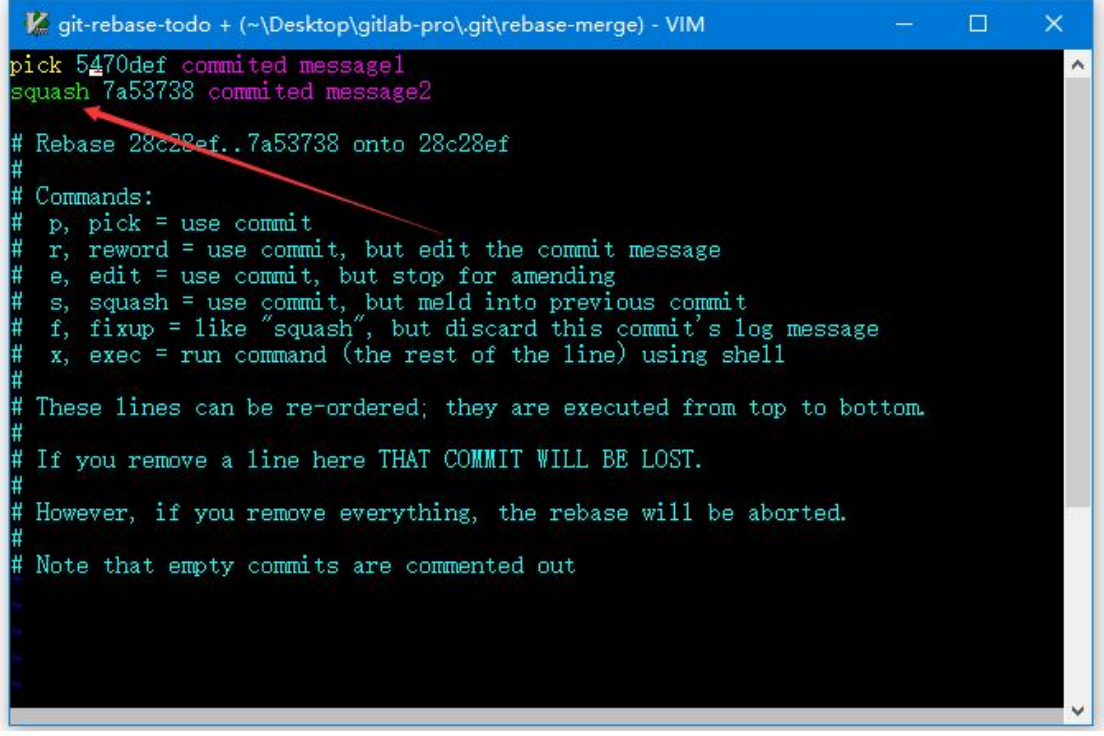
```
$ git commit . -m "committed message2"
```

步骤(3): 现在, 使用以下命令将上述两个提交压缩到一个提交中:

```
$ git rebase -i HEAD~2
```

这里, `git rebase` 命令用于从一个分支集成到另一个分支, `HEAD~2` 指定最后两个压缩的提交, 如果要压缩四个提交, 则需要将其写为 `HEAD~4`。 更重要的一点是, 需要至少两次提交来完成压缩操作。

步骤(4): 输入上述命令后, 它将打开下面的编辑器, 必须在其中更改第二行的 pick 单词为 squash(需要压缩此提交)。

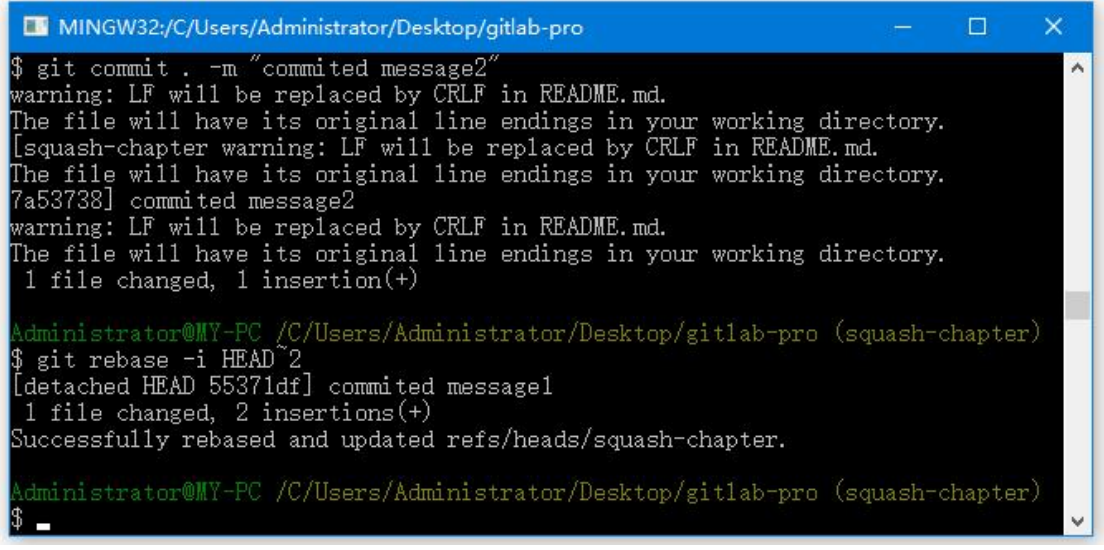


```
git-rebase-todo + (~\Desktop\gitlab-pro\git\rebase-merge) - VIM
pick 5470def committed message1
squash 7a53738 committed message2

# Rebase 28c28ef..7a53738 onto 28c28ef
#
# Commands:
# p, pick = use commit
# r, reword = use commit, but edit the commit message
# e, edit = use commit, but stop for amending
# s, squash = use commit, but meld into previous commit
# f, fixup = like "squash", but discard this commit's log message
# x, exec = run command (the rest of the line) using shell
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
# Note that empty commits are commented out
#
#
#
```

完成更改之后, 按 Esc 键, 然后按冒号(:)并键入 wq 保存并退出屏幕。

步骤(5): 现在将分支推送到远程存储库, 如下所示:



```
MINGW32:/C/Users/Administrator/Desktop/gitlab-pro
$ git commit . -m "committed message2"
warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.
[squash-chapter warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.
7a53738] committed message2
warning: LF will be replaced by CRLF in README.md.
The file will have its original line endings in your working directory.
1 file changed, 1 insertion(+)

Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (squash-chapter)
$ git rebase -i HEAD~2
[detached HEAD 55371df] committed message1
1 file changed, 2 insertions(+)
Successfully rebased and updated refs/heads/squash-chapter.

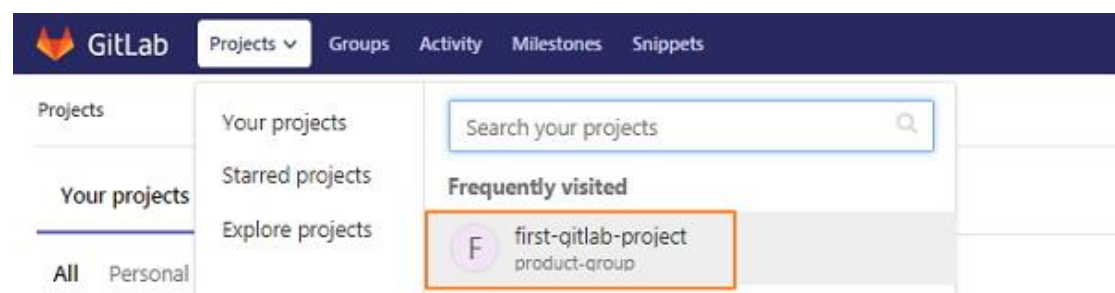
Administrator@MY-PC /C/Users/Administrator/Desktop/gitlab-pro (squash-chapter)
$
```

GitLab 问题跟踪器

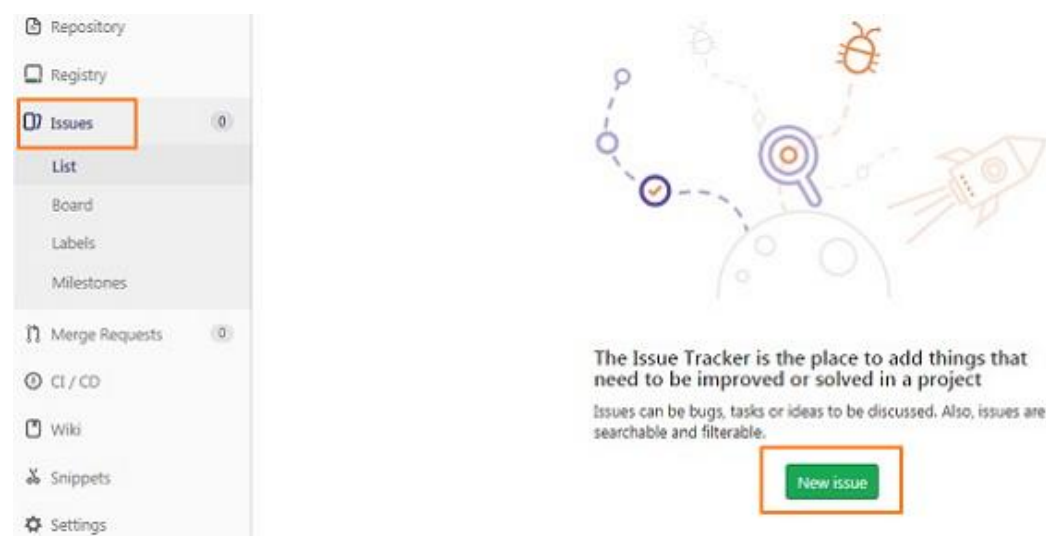
GitLab 创建问题

在本章中，我们将讨论如何在项目中创建问题：

步骤(1)：登录到您的 GitLab 帐户并转到项目部分下的项目：



步骤(2)：转到问题选项卡并单击新问题按钮创建一个新问题，如下所示：



步骤(3)：现在，填写标题，说明等信息，如果需要，可以选择用户分配问题，里程碑(请参阅本章以获取更多信息)，操作时标签或可由开发人

员在创建之后自行选择。

The screenshot shows the 'New Issue' form in GitLab. On the left is a sidebar with navigation links: Overview, Registry, Issues (selected), List, Board, Labels, Milestones, Merge Requests, CI/CD, Wiki, Snippets, and Settings. The main form has a 'Title' field with 'Issue Demo'. Below it is a 'Description' field with a 'Write' tab and a 'Preview' tab. The description text is 'It contains unwanted text...'. There are also fields for 'Assignee' (set to 'Unassigned'), 'Milestone' (set to 'Milestone'), and 'Labels'. A green 'Submit issue' button is highlighted with a red box at the bottom left of the form.

步骤(4): 点击 Submit issue 按钮，您将看到一个问题的概述以及标题和描述，如下所示：

The screenshot shows the 'Issue Demo' page in GitLab. The page has a sidebar with navigation links: Registry, Issues (selected), List, Board, Labels, Milestones, Merge Requests, CI/CD, Wiki, Snippets, and Settings. The main content area shows the issue title 'Issue Demo' and the description 'It contains unwanted text...'. There are buttons for 'Create merge request' and 'Close issue'. A red box highlights the 'Close issue' button.

GitLab 合并请求

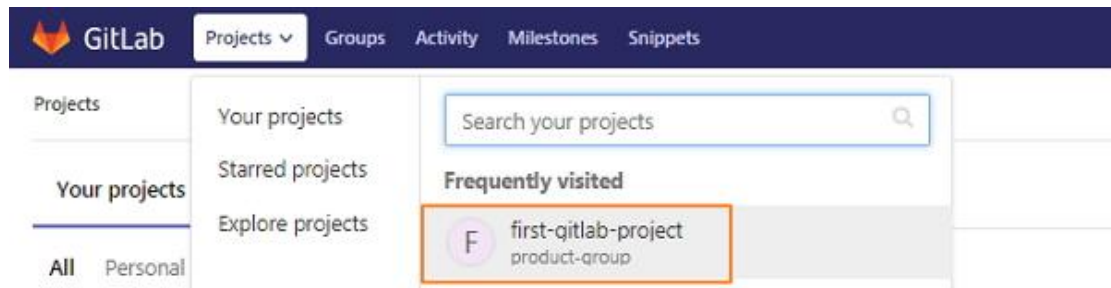
合并请求可用于对项目其他人员之间所做的代码进行交换，与他们讨论更改。

合并请求的步骤

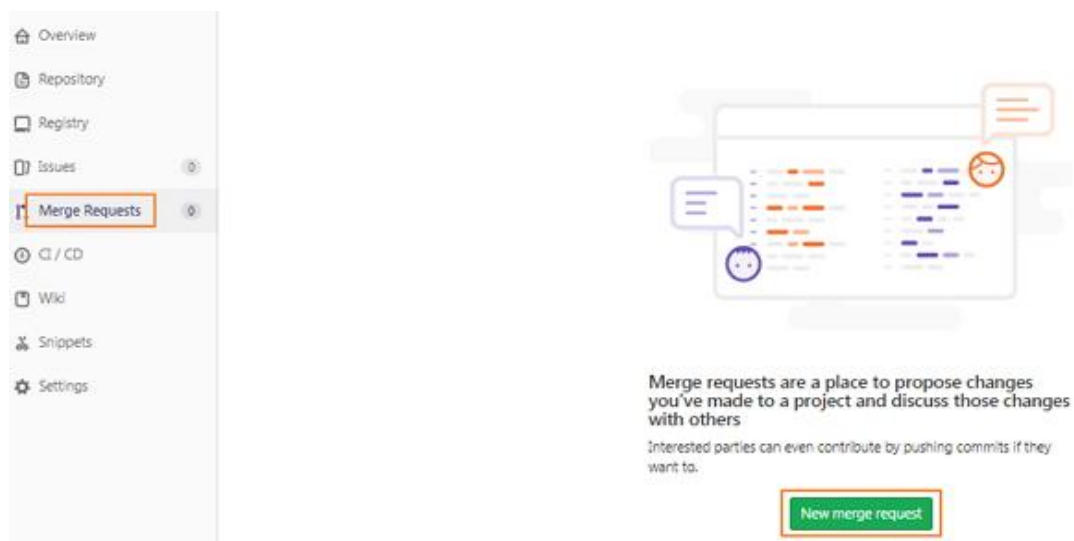
步骤(1): 在创建新的合并请求之前, 应该在 GitLab 中创建一个分支。

您可以参考[本章](#)创建分支:

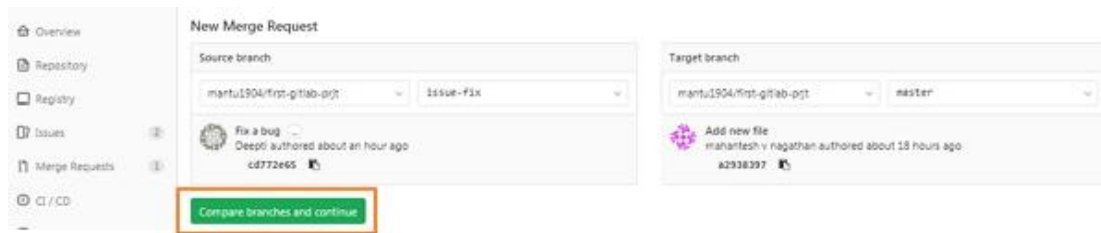
步骤(2): 登录到您的 GitLab 帐户并转到项目部分下的项目:



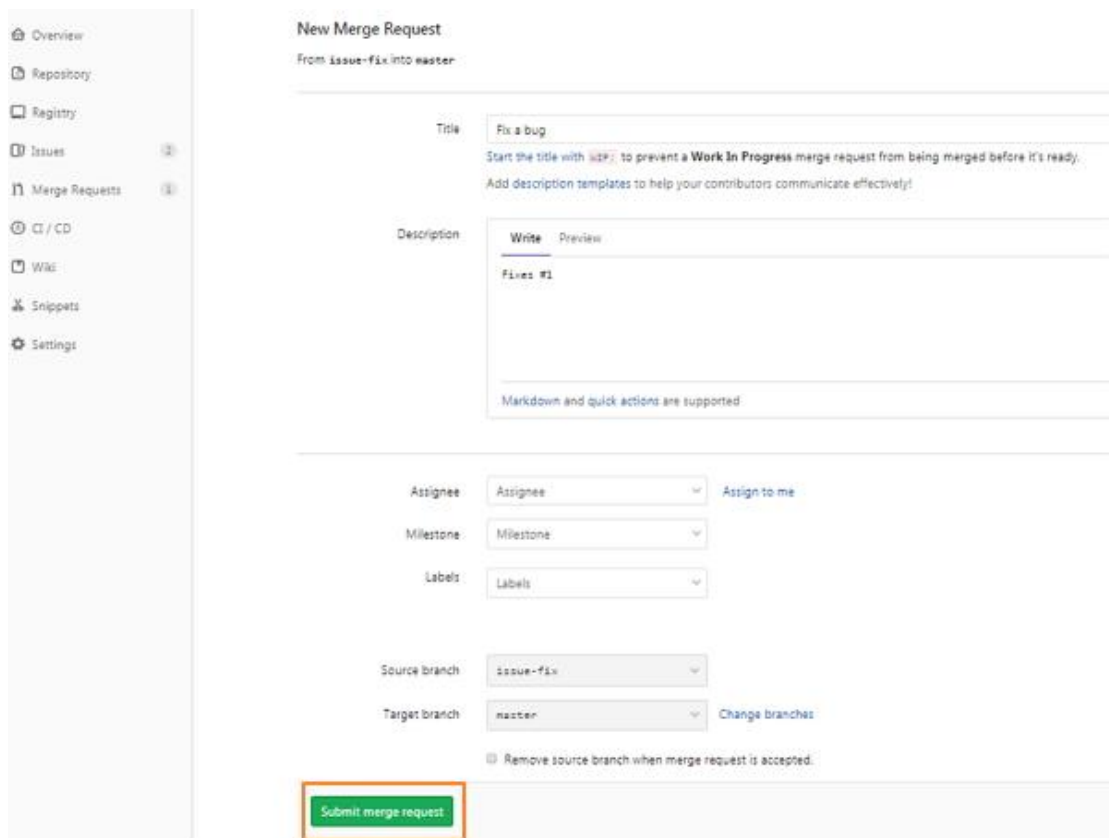
步骤(3): 点击 Merge Requests 选项卡, 然后点击 New merge request 按钮:



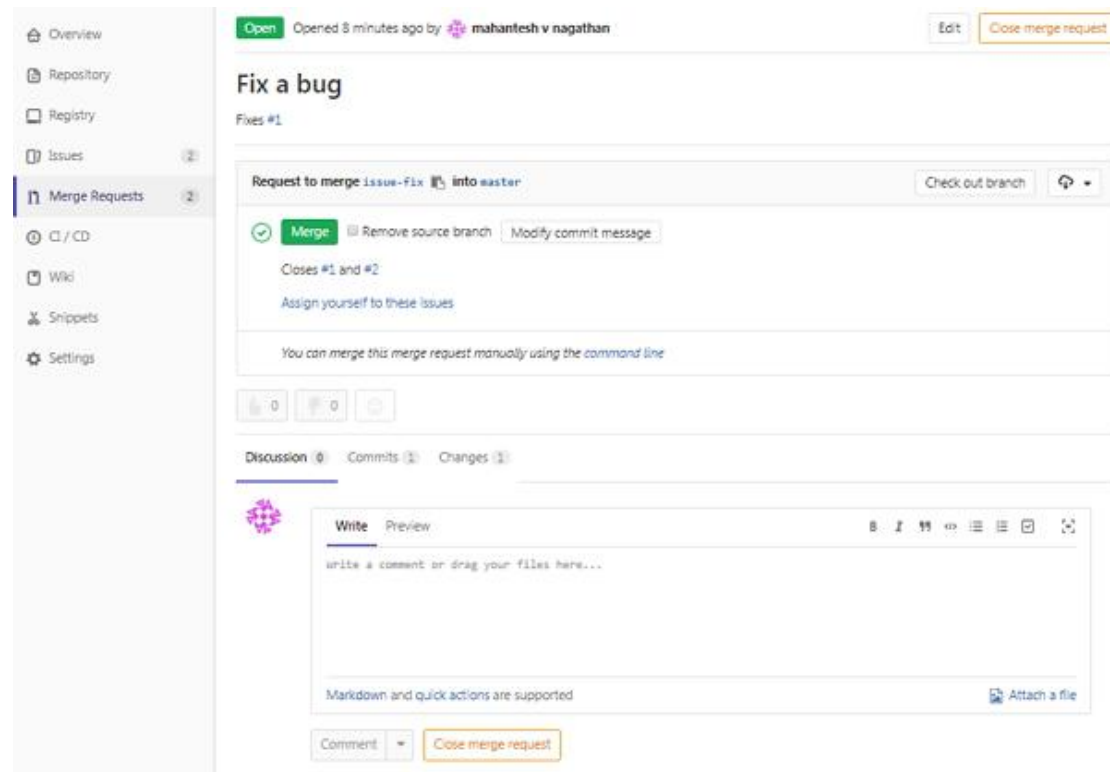
步骤(4): 要合并请求, 请从下拉列表中选择源分支和目标分支, 然后单击 Compare branches and continue 按钮, 如下所示:



步骤(5): 您将看到标题, 说明和其他字段, 如分配用户, 设置里程碑, 标签, 源分支名称和目标分支名称, 然后单击提交合并请求按钮:



步骤(6): 提交合并请求后, 您将获得一个新的合并请求屏幕, 如下所示:

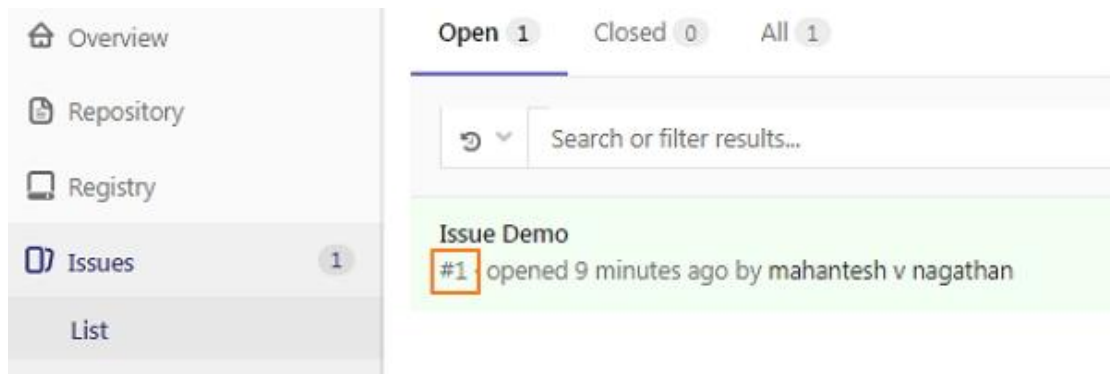


GitLab 合并请求

GitLab 可以引用提交消息中的特定问题来解决特定的问题。在本章中, 我们将讨论如何在 GitLab 中引用问题:

步骤(1): 要引用问题, 您需要创建问题的问题编号。 要创建问题, 请参阅[创建问题](#)章节。

步骤(2): 要查看创建的问题, 请单击 Issues 选项卡下的 List 选项:



步骤(3): 在对本地存储库进行更改之前, 请使用以下命令检查它是否为最新版本:

```
$ git checkout master && git pull
```

git pull 命令从远程服务器下载最新的更改并直接集成到当前工作文件中。

步骤(4): 现在, 使用 git checkout 命令创建一个名为 issue-fix 的新分支:

```
$ git checkout -b issue-fix
```

步骤(5): 现在, 将一些内容添加到 README.md 文件中以修复该错误:

```
$ echo "fix this bug" >> README.md
```

步骤(6): 使用以下命令输入上述更改的提交消息:

```
$ git commit -a
```

此命令将打开下面的屏幕，然后按键盘上的插入键为 issue-fix 分支添加提交消息。

```
Fixes #1_
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
# On branch issue-fix
# Changes to be committed:
#   modified:   README.md
#
# Untracked files:
#   READMEFile.md
#
--
C:\first-gitlab-project\.git\COMMIT_EDITMSG[+] [dos] <15:52 26/02/2018
-- INSERT --
```

现在按 Esc 键，然后按冒号(:)键入 wq 保存并退出屏幕。

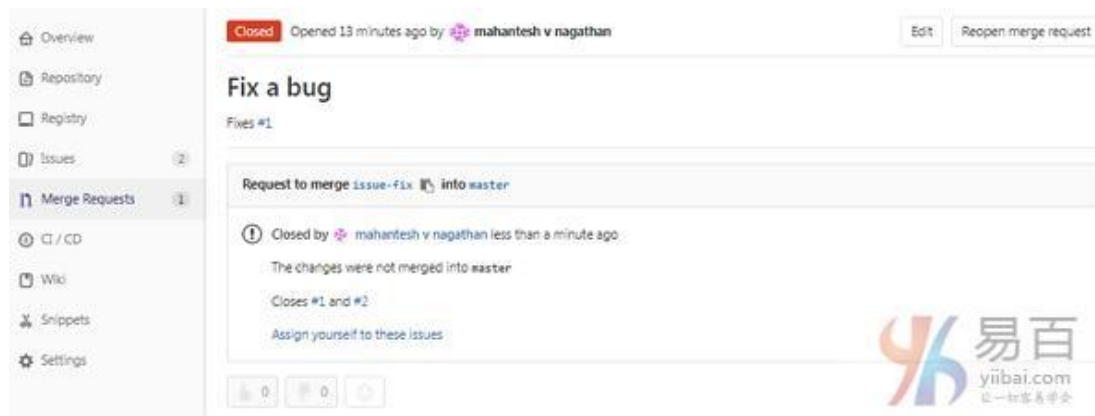
步骤(7): 现在使用以下命令将分支推送到远程存储库:

```
$ git push origin issue-fix
```

```
C:\first-gitlab-project>git push origin issue-fix
Username for 'https://gitlab.com': mantu1904
Password for 'https://mantu1904@gitlab.com':
Counting objects: 8, done.
Delta compression using up to 4 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 294 bytes | 0 bytes/s, done.
Total 3 (delta 0), reused 0 (delta 0)
remote:
remote: To create a merge request for issue-fix, visit:
remote:   https://gitlab.com/mantu1904/first-gitlab-project/merge_requests/new?merge_request%5Bsource_branch%5D=issue-fix
remote:
To https://gitlab.com/mantu1904/first-gitlab-project.git
 * [new branch]      issue-fix -> issue-fix
```

步骤(8): 登录到您的 GitLab 帐户并创建一个新的合并请求。可以参考[合并请求](#)章节来创建合并请求。

步骤(9): 创建合并请求后, 您将被重定向到合并请求页面。 当点击 Close merge request 按钮时(参考[合并请求](#)章节的步骤(6)中的截图), 将在关闭合并请求后看到关闭选项。

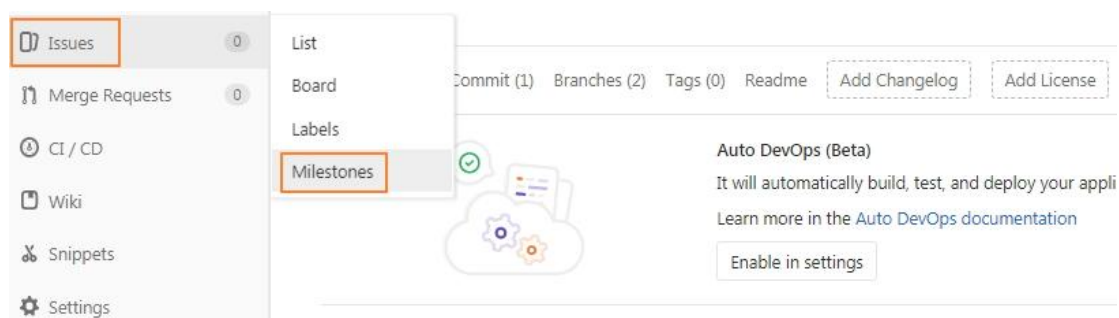


GitLab 里程碑

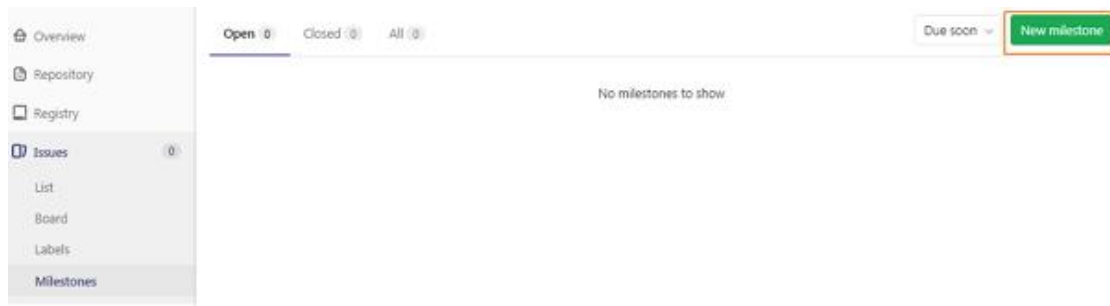
里程碑用于安排问题并将请求合并到一个确定的组中, 通过设置开始日期和到期日期, 可以在指定的时间内完成。

创建里程碑的步骤

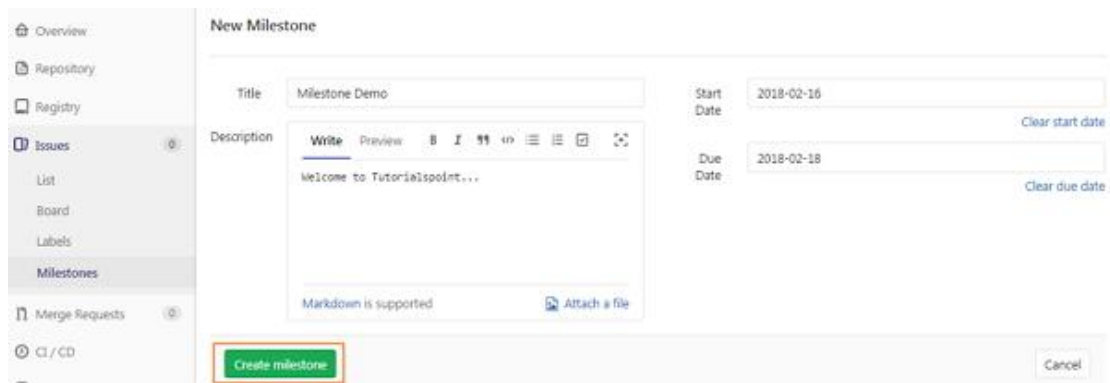
步骤(1): 登录到您的 GitLab 帐户, 转到您的项目并单击问题选项卡下的 Milestones 选项:



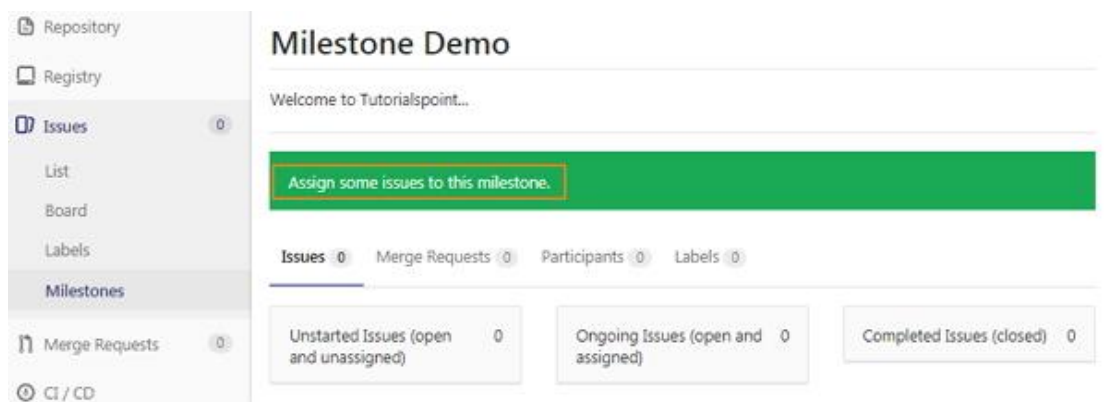
步骤(2): 点击 New milestone 按钮:



步骤(3): 现在输入标题, 说明, 开始日期和截止日期, 然后单击 Create milestone 按钮, 如下图所示:



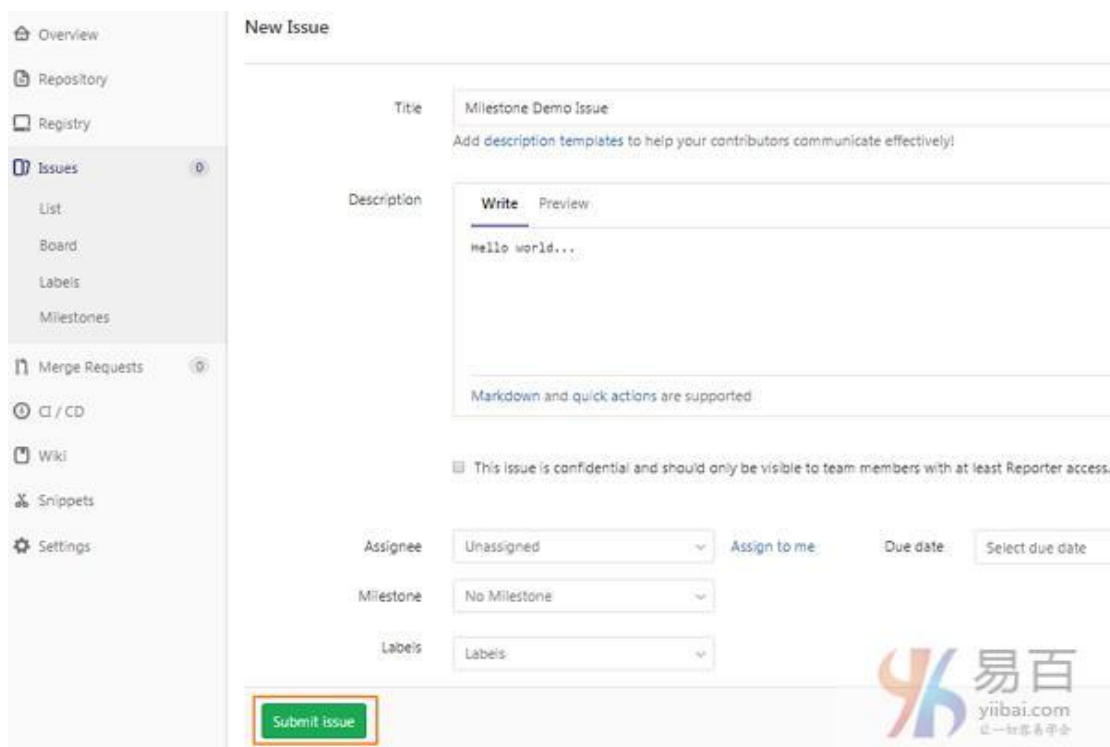
步骤(4): 创建一个里程碑后, 它会显示一条消息, 指出“为此里程碑分配一些问题”, 如下所示:



步骤(5): 现在转到 Issues 选项卡, 然后单击 New issue 按钮为里程碑创建问题:



步骤(6): 现在, 填写标题, 说明等信息, 如果需要, 可以选择用户在操作时指定问题, 里程碑, 标签或稍后由开发人员自行选择。 点击 Submit issue 按钮。



步骤(7): 创建问题后，您将看到问题以及标题和说明。 在右侧，单击 Edit 选项并为 Milestone 部分下的问题分配里程碑：



步骤(8): 现在回到里程碑部分，您将看到添加的里程碑以及创建的问题：



GitLab 维基页面

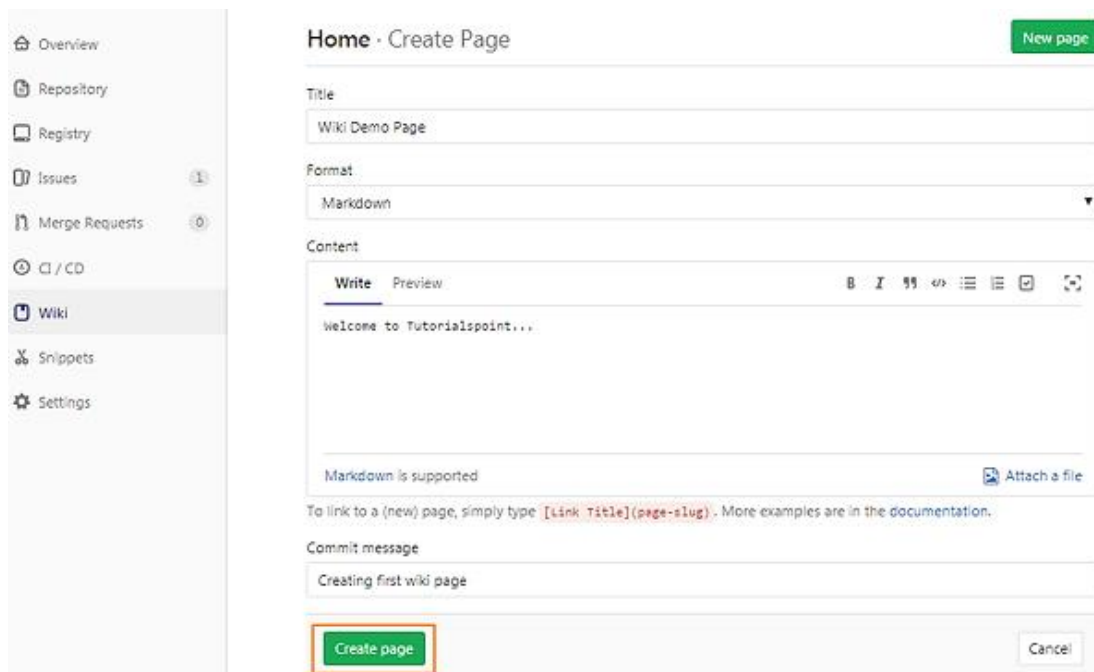
Wiki 是一个在 GitLab 中维护项目文档的系统。 它就像一个维基百科，它可以编辑和授权管理维基页面。 访客可以查看维基页面，开发者可以创建和编辑维基页面。

创建 Wiki 页面的步骤

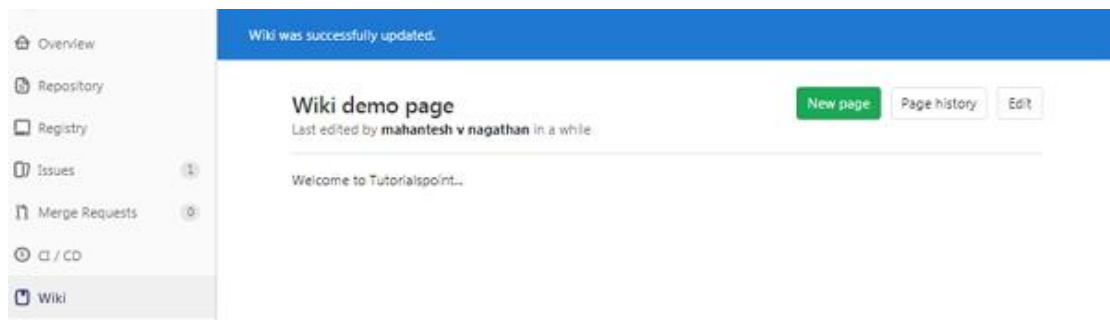
步骤(1): 登录到您的 GitLab 帐户，转到您的项目并单击 Wiki 选项卡:



步骤(2): 现在输入标题，格式，填写内容部分，添加提交消息，然后点击创建页面按钮:



步骤(3): 您将获得新创建的维基页面，如下图所示:



Gitlab 用户在组里面有 5 种不同权限:

Guest: 可以创建 issue、发表评论, 不能读写版本库 **Reporter**: 可以克隆代码, 不能提交, QA、PM

可以赋予这个权限 **Developer**: 可以克隆代码、开发、提交、push, 普通开发可以赋予这个权限

Maintainer: 可以创建项目、添加 tag、保护分支、添加项目成员、编辑项目, 核心开发可以赋予这个

权限 **Owner**: 可以设置项目访问权限 - Visibility Level、删除项目、迁移项目、管理组成员, 开发组组长

可以赋予这个权限

gitlab 访问权限 - Visibility Level

Private - 私有, 只有属于该项目成员才有原先查看

Internal - 内部, 用个 Gitlab 账号的人都可以 clone

Public - 公开, 任何人可以 clone 请注明出处。